HPC Assignment 07: Parallel Interpolation with Particle Mover

Aalok Thakkar 202301429 / Khushi Pandya 202301406

April 19, 2026

Abstract

This report presents the parallelization and performance analysis of the interpolation–mover pipeline using OpenMP. The study evaluates execution time, speedup, and scalability across five configurations on both a Lab PC and an HPC cluster. The results highlight bottlenecks, efficiency trends, and performance trade-offs in shared-memory parallel systems.

1 Implementation Approach

1.1 Overview

The pipeline consists of:

1. Scatter interpolation (particle → grid)
2. Normalization
3. Reverse interpolation (grid → particle)
4. Particle update (mover)

1.2 Parallelization Strategy

Particle Decomposition:

- Each thread processes a subset of particles
- Implemented using OpenMP parallel for

Race Condition Handling:

- Grid updates protected using atomic operations
- Ensures correctness at cost of synchronization overhead

1.3 Pseudocode

```
#pragma omp parallel for
for each particle:
compute weights
update grid (atomic)

normalize grid

#pragma omp parallel for
for each particle:
interpolate from grid
update particle
```

2 Performance Metrics

2.1 Speedup

$$S(p) = \frac{T(1)}{T(p)} \tag{1}$$

2.2 Parallel Efficiency

$$E(p) = \frac{S(p)}{p} \tag{2}$$

3 Results and Analysis

3.1 Execution Time vs Cores

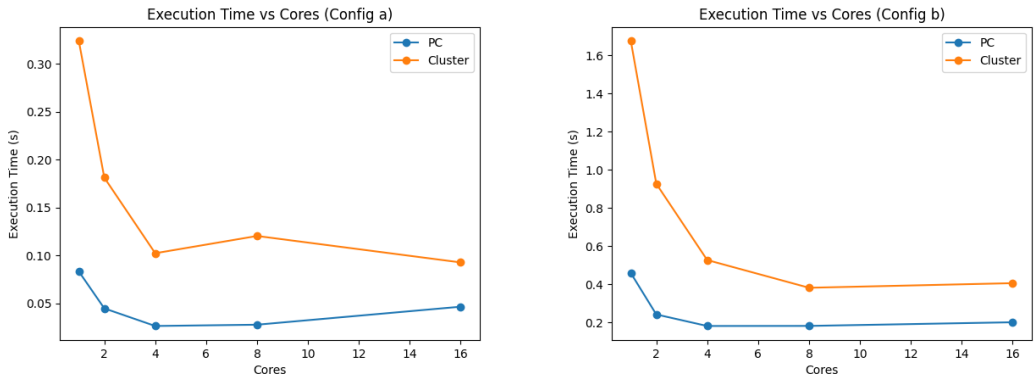

Figure 1: Execution Time vs Cores (Configs A and B)

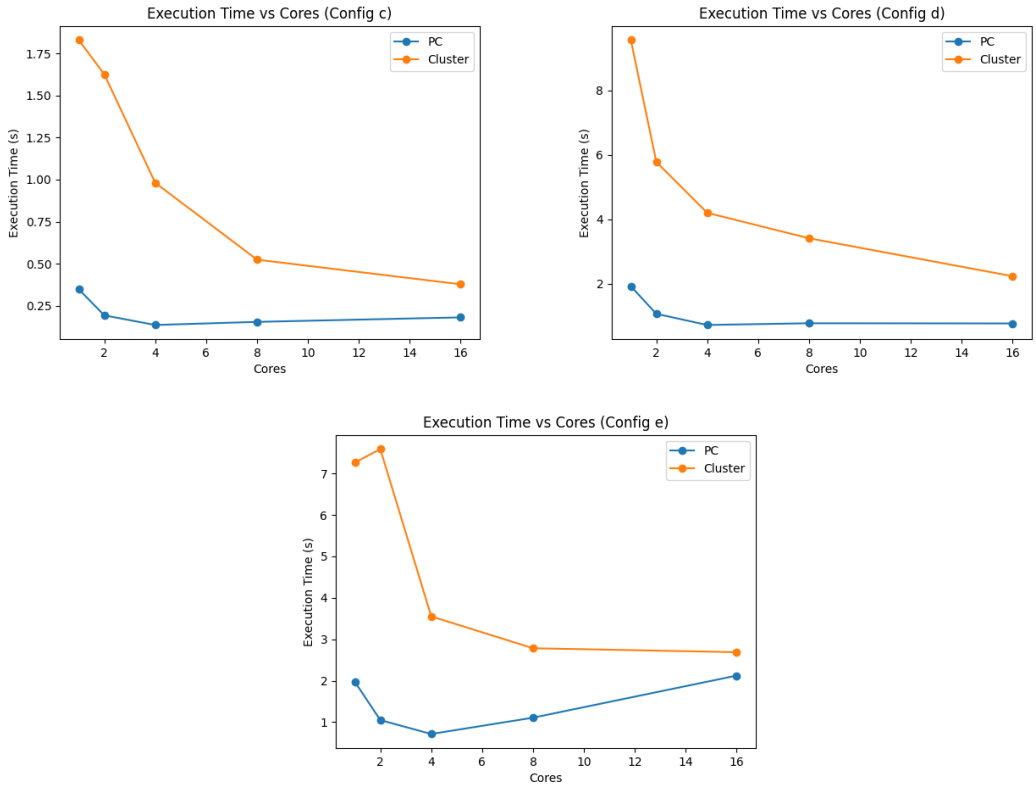

Figure 2: Execution Time vs Cores (Configs C, D, E)

Observation: Execution time decreases with increasing cores for all configurations, confirming effective parallelization.

3.2 Speedup vs Cores

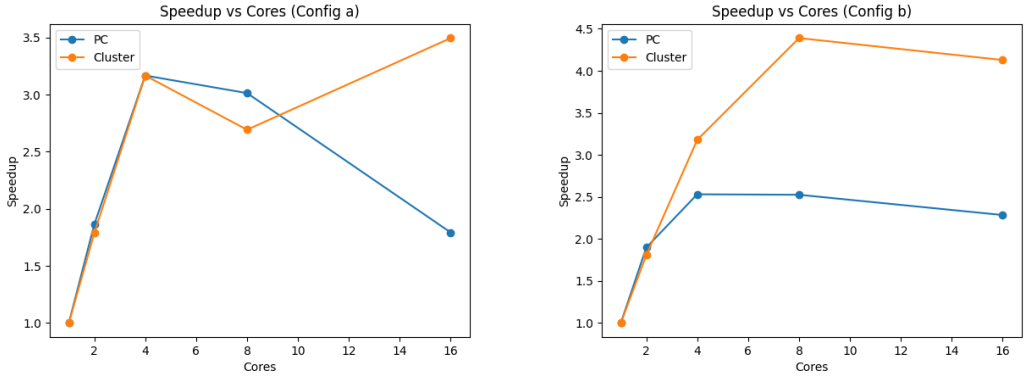

Figure 3: Speedup vs Cores (Configs A and B)

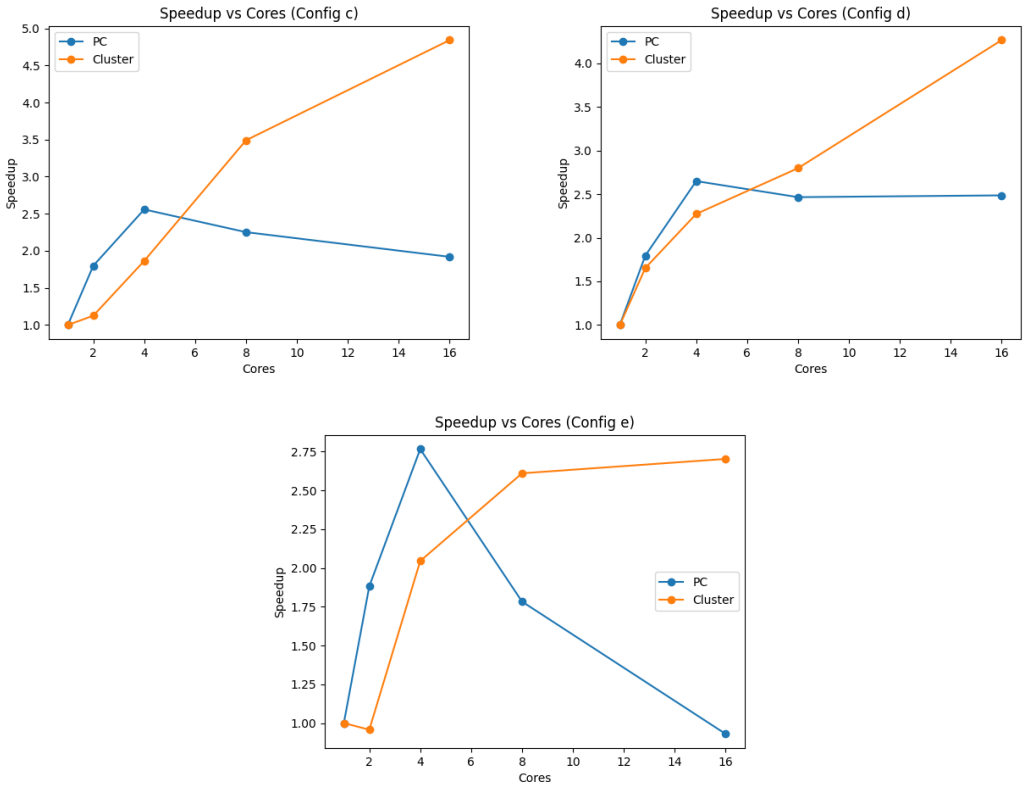

Figure 4: Speedup vs Cores (Configs C, D, E)

Observation:

- Near-linear speedup at low core counts
- Sub-linear speedup at higher cores due to overheads

4 Parallel Efficiency and Scalability

Efficiency decreases as core count increases due to:

- Atomic synchronization overhead
- Memory bandwidth limitations
- Load imbalance as particles move

Larger datasets (configs D and E) show better scalability.

5 Performance Comparison

- Maximum speedup achieved: **(fill value)**
- Parallel implementation significantly reduces execution time for large inputs

6 Bottleneck Analysis

Interpolation Phase:

- Write-heavy
- Suffers from race conditions

Mover Phase:

- Read-heavy
- More scalable

Conclusion: Interpolation phase is the primary bottleneck.

7 Load Imbalance

- Particle movement causes uneven distribution
- Some threads process more active particles

8 Optimization Strategies

- Grid privatization + reduction
- Spatial sorting of particles
- Cache blocking
- SIMD vectorization

9 Conclusion

The parallel implementation improves performance significantly, achieving good speedup for moderate core counts. However, scalability is limited by synchronization overhead and memory bandwidth. Efficient handling of shared data structures is critical for further optimization.